

Intelligence Evaluation Principles

Our evaluation approach is guided by four core principles:

- **Standardized:** All models are evaluated under identical conditions with consistent prompting strategies, temperature settings, and evaluation criteria.
- **Unbiased:** We employ evaluation techniques that avoid unfairly penalizing models for answers that correctly follow the instructions in our prompts. This includes using clear prompts, robust answer extraction methods, and flexible answer validation to accommodate valid variations in model outputs.
- **Zero-Shot Instruction Prompted:** We evaluate using clear instructions without examples or demonstrations, testing models' ability to follow directions without few-shot learning. This approach aligns with modern instruction-tuned and chat style models.
- **Transparent:** We disclose our methodology, including prompt templates, evaluation criteria, and limitations.

General Testing Parameters

We test all evals with the following settings:

- **Temperature:** 0 for non-reasoning models, 0.6 for reasoning models (unless another temperature is recommended by the model lab)
- **Maximum output tokens:**
 - Non-reasoning models: 16,384 tokens (adjusted downward where models have a smaller context window, or lower maximum output tokens cap)
 - Reasoning models: Maximum output tokens allowed, as disclosed by model creators (custom setting for each reasoning model)
- **Error handling:**
 - Automatic retry on API failures (up to 30 attempts)
 - All questions that failed all 30 retries are manually reviewed. Results where persistent API failures have caused issues are not published. Errors where all available APIs for proprietary models block a certain question may lower scores (this effect is not material)
- **Scoring methodology:** We generally use pass@1 scoring across our evaluations, where a model must produce the correct answer on its first attempt. For evaluations with multiple repeats, pass@1 is calculated by aggregating results across all repeats. This is calculated as:
 - $pass@1 = \frac{1}{k} \sum_{i=1}^k p_i$
 - $p_i = \frac{1}{k} \sum_{j=1}^k \mathbb{1}_{\{y_j = y^*\}}$
 - k
 - 1
 - $i=1$

- k
- p
- i
-
- where $p_i = 1$ if attempt i is correct, 0 otherwise, and k is the total number of test instances across all repeats.

We maintain internal copies of all evaluation datasets. The sources of our selected datasets are listed below.

For the Artificial Analysis Intelligence Index evaluations, we use the token counts reported by each model's API provider where available, to accurately report the cost of running the Intelligence Index. In the rare cases where provider token counts are unavailable, we use a canonical-tokenizer fallback. This contrasts with the approach in performance benchmarking, where we use client-side token counts from the `o200k_base` tokenizer to standardize token counts for the same text across models. When reporting cache hit rates and cost, we combine these token counts with live measurements of the model's typical cache hit rate, rather than relying on the one-off measurement when the evaluation was run.

We use [e2b](#) as our primary sandbox provider for agentic benchmarks.

Prompt Templates, Answer Extraction and Evaluation

Multiple Choice Questions (GPQA, MMLU-Pro)

We prompt multi-choice evals with the following instruction prompt. This prompt was independently developed by Artificial Analysis, and carefully validated with various ablation studies. We assess that this prompt is a clearer, and therefore fairer, approach than traditional completion-style multi-choice evaluation methodologies or other instruction prompts we tested.

GPQA uses four options (A–D). MMLU-Pro uses ten options (A–J); we use the same structure with additional choices.

Answer the following multiple choice question. The last line of your response should be in the following format: 'Answer: A/B/C/D' (e.g. 'Answer: A').

{Question}

A) {A}

B) {B}

C) {C}

D) {D}

Answer the following multiple choice question. The last line of your response should be in the following format: 'Answer: A/B/C/D/E/F/G/H/I/J' (e.g. 'Answer: A').

{Question}

A) {A}

B) {B}

C) {C}

D) {D}

E) {E}

F) {F}

G) {G}

H) {H}

I) {I}

J) {J}

Multiple Choice Extraction Regex

We extract multiple choice answers using a multi-stage approach to handle various answer formats.

For single-letter responses, we use the letter directly. Otherwise, we first attempt to match our primary pattern that looks for formal "Answer: X" format (accounting for optional markdown formatting):

Primary Pattern:

```
(?i) [\*\_\]{0,2}Answer[\*\_\]{0,2}\s*:[\s\*\_\]{0,2}\s*([A-Z]) (?![a-zA-Z0-9])
```

If the primary pattern fails, we attempt the following fallback patterns in sequence to catch various answer formats:

- LaTeX boxed notation (e.g., `\boxed{A}` or `\boxed{The answer is A}`)
`\boxed\{ [^\}]* ([A-Z]) [^\}]* \}`
- Natural language (e.g., "answer is B")
`answer is ([a-zA-Z])`
- With parenthesis (e.g., "answer is (C)")
`answer is \\\{([a-zA-Z])\}`
- Choice format (e.g., "D) some answer text")
`([A-Z])\)\s*[^A-Z]*`
- Explicit statement (e.g., "E is the correct answer")
`([A-Z])\s+is\s+the\s+correct\s+answer`
- Standalone letter at end of response
`([A-Z])\s*$`
- Letter followed by period (e.g., "F.")
`([A-Z])\s*\.`
- Letter followed by non-word character
`([A-Z])\s*[^w]`

We always take the last match found to account for self-correction in responses.

Equality Checker LLM

For evaluations with open-ended answers (HLE, AA-LCR), we use an equality checker LLM to determine if a model's response is semantically equivalent to the correct answer. This approach uses a language model to assess whether two answers have the same meaning, even if phrased differently. The equality checker evaluates semantic equivalence rather than requiring exact string matches, which is particularly important for questions where multiple valid phrasings exist.

HLE and AA-LCR share a single equality checker, GPT-5.6 Luna (medium), selected based on validation against human judgement. Specific equality checker prompts for each evaluation are detailed in the sections below.

HLE (Humanity's Last Exam)

We prompt HLE with the following text as a system prompt, with a difference between Exact Answer and Multiple Choice questions - these prompts are used with no changes from the original implementation by Hendrycks et al. For models that do not support a system prompt, we prepend the system prompt to the user prompt.

System Prompt for Exact Answer Questions:

"Your response should be in the following format:

Explanation: {your explanation for your final answer}

Exact Answer: {your succinct, final answer}

Confidence: {your confidence score between 0% and 100% for your answer}"

System Prompt for Multiple Choice Questions:

"Your response should be in the following format:

Explanation: {your explanation for your answer choice}

Answer: {your chosen answer}

Confidence: {your confidence score between 0% and 100% for your answer}"

[HLE Equality Checker Prompt](#)

We use the following prompt on GPT-5.6 Luna (medium) to check if the response answer is correct - this prompt is again used with no changes from the original implementation by Hendrycks et al.

Judge whether the following [response] to [question] is correct or not based on the precise and unambiguous [correct_answer] below.

[question]: {question}

[response]: {response}

Your judgement must be in the format and criteria specified below:

extracted_final_answer: The final exact answer extracted from the [response]. Put the extracted answer as 'None' if there is no exact, final answer to extract from the response.

[correct_answer]: {correct_answer}

reasoning: Explain why the extracted_final_answer is correct or incorrect based on [correct_answer], focusing only on if there are meaningful differences between [correct_answer] and the extracted_final_answer. Do not comment on any background to the problem, do not attempt to solve the problem, do not argue for any answer different than [correct_answer], focus only on whether the answers match.

correct: Answer 'yes' if extracted_final_answer matches the [correct_answer] given above, or is within a small margin of error for numerical problems. Answer 'no' otherwise, i.e. if there if there is any inconsistency, ambiguity, non-equivalency, or if the extracted answer is incorrect.

confidence: The extracted confidence score between 0|\%| and 100|\%| from [response]. Put 100 if there is no confidence score available.

LCR Equality Checker Prompt

Assess whether the following CANDIDATE ANSWER is CORRECT or INCORRECT.
For the CANDIDATE ANSWER to be correct, it must be consistent with the OFFICIAL ANSWER.

The question, for reference only: {question}

The OFFICIAL ANSWER: {official_answer}

CANDIDATE ANSWER TO ASSESS: {candidate_answer}

Reply only with CORRECT or INCORRECT.

Mathematical Questions (AIME 2025)

We prompt AIME with the following instruction prompt:

Solve the following math problem step by step. Put your answer inside \\boxed{{}}.

{Question}

Remember to put your answer inside \\boxed{{}}.

Mathematical Equality Checker Prompt

As described above, we supplement our script-based grading with a language model equality checker. We use the following prompt with Llama 3.3 70B to check if two answers are equivalent. This prompt was developed by OpenAI and released in their simple-evals repository.

Look at the following two expressions (answers to a math problem) and judge whether they are equivalent. Only perform trivial simplifications

Examples:

Expression 1: $2x + 3$

Expression 2: $3 + 2x$

Yes

Expression 1: $3/2$

Expression 2: 1.5

Yes

Expression 1: $x^2 + 2x + 1$

Expression 2: $y^2 + 2y + 1$

No

Expression 1: $x^2 + 2x + 1$

Expression 2: $(x + 1)^2$

Yes

Expression 1: $3245/5$

Expression 2: 649

No

(these are actually equal, don't mark them equivalent if you need to do nontrivial simplifications)

Expression 1: $2/(-3)$

Expression 2: $-2/3$

Yes

(trivial simplifications are allowed)

Expression 1: 72 degrees

Expression 2: 72

Yes

(give benefit of the doubt to units)

Expression 1: 64

Expression 2: 64 square feet

Yes

(give benefit of the doubt to units)

YOUR TASK

Respond with only "Yes" or "No" (without quotes). Do not include a rationale.

Expression 1: %(expression1)s

Expression 2: %(expression2)s

[Code Generation Tasks](#)

[SciCode](#)

We prompt SciCode with the following prompt, used with no changes from the original implementation of the Scientist Annotated Background prompt by Tian et al.

PROBLEM DESCRIPTION:

You will be provided with problem steps along with background knowledge necessary for solving the problem. Your task will be to develop a Python solution focused on the next step of the problem-solving process.

PROBLEM STEPS AND FUNCTION CODE:

Here, you'll find the Python code for the initial steps of the problem-solving process. This code is integral to building the solution.

```
{problem_steps_str}
```

NEXT STEP - PROBLEM STEP AND FUNCTION HEADER:

This part will describe the next step in the problem-solving process. A function header will be provided, and your task is to develop the Python code for this next step based on the provided description and function header.

```
{next_step_str}
```

DEPENDENCIES:

Use only the following dependencies in your solution. Do not include these dependencies at the beginning of your code.

```
{dependencies}
```

RESPONSE GUIDELINES:

Now, based on the instructions and information provided above, write the complete and executable Python program for the next step in a single block.

Your response should focus exclusively on implementing the solution for the next step, adhering closely to the specified function header and the context provided by the initial steps.

Your response should NOT include the dependencies and functions of all previous steps. If your next step function calls functions from previous steps, please make sure it uses the headers provided without modification.

DO NOT generate EXAMPLE USAGE OR TEST CODE in your response. Please make sure your response python code in format of ```python```.

LiveCodeBench

We prompt LiveCodeBench with the following prompt, used with no changes from the original implementation of the LiveCodeBench prompt by the original team. We note, however, that we do not apply the custom system prompts that the LiveCodeBench team uses - we do not use their generic sytem prompt nor their custom system prompts for certain models.

Questions with starter code:

Question:

{question.question_content}

Format: You will use the following starter code to write the solution to the problem and enclose your code within delimiters.

```python

{question.starter\_code}

```

Answer: (use the provided format with backticks)

Questions without starter code:

Question:

{question.question_content}

Format: Read the inputs from stdin solve the problem and write the answer to stdout (do not directly test on the sample inputs). Enclose your

code within delimiters as follows. Ensure that when the python program runs, it reads the inputs, runs the algorithm and writes output to STDOUT.

```
```python
```

```
YOUR CODE HERE
```

```
```
```

Answer: (use the provided format with backticks)

[Code Extraction Regex](#)

We extract the code from the response using the following regex:

```
(?<=```python\n) ((?:\n|.)+?) (?:\n```)
```